

团队章程文档

团队名称：第67组

日期：2026年4月13日

一 角色分配计划

需求负责人（负责需求获取、文档撰写、用例建模）

陆俊达

对所承担角色的理解

作为这个项目的需求负责人，我的核心理解是：我守护的是“为什么做”和“做什么”，而不是“怎么做”。校园互助这个概念听起来很轻巧，但落地的坑极多。我的职责就是在大家头脑发热要堆功能之前，先冷静地把真正的战场画出来。校园环境不同于社会，这里是一个半熟人社群，纯粹的利益驱动会死，纯粹的爱心驱动也会死。我要做的就是在这两者之间，找到一个能让同学放下戒备、愿意顺手点一下的平衡点。

该角色的核心职责与目标

我的核心职责是做减法。开发团队和设计同学会有无数灵光一现的点子，比如实时聊天、信用分体系、甚至社交动态。但我的工作是要拿着用户最痛的场景去质问：加了这个功能，是让取快递的人更省心了，还是让顺路带饭的人操作更麻烦了？我要确保团队有限的精力只消耗在唯一的核心命门上——极低门槛的供需匹配。我要挡掉所有会拉长用户决策时间、增加心理负担的复杂设计，哪怕它看起来很美。至于目标，在这个阶段我追求的不是平台的“大而全”，而是“顺”。我衡量成功标准非常朴素：当一个同学站在快递站门口，看着长长的队伍，打开我们的小程序，能在三次点击内找到一个顺路回宿舍楼且值得信任的人。只要能让这种“刚好顺手”缘分大规模发生，这个平台就有了在校园里活下去的根基。我的目标就是让第一批用户用完之后，心里想的是“真方便”，而不是“好麻烦，还不如我自己跑一趟”。

架构负责人（负责技术选型、架构设计、ADR 撰写）

孙博一

对所承担角色的理解

我认为架构负责人不只是完成技术选型与绘制架构图，更是项目技术体系的设计者、技术风险的防控者与产品长期可迭代性的守护者。

该角色的核心职责与目标

我的核心职责是基于校园互助平台的业务需求与团队现状，设计简洁、健壮、低耦合的系统架构，合理划分模块边界，明确技术栈、数据结构与交互规范，同时保障性能、安全、可扩展性等非功能需求落地。我需要提前预判技术瓶颈与潜在风险，为开发团队提供清晰的技术标准与实施依据，确保系统结构合理、稳定可靠、易于维护与后续扩展，支撑产品从MVP平稳走向长期迭代，为整个项目提供坚实的技术底座。

开发负责人（负责编码规范、代码审查、核心模块开发）

刘渝川

对所承担角色的理解

我对这个角色的理解是，开发负责人不仅仅是负责编写代码的人，更是负责推动系统从需求走向实现的人。这个角色需要把前期的需求分析和设计内容转化为可执行的开发任务，保证团队在编码阶段能够有序推进，并且尽量减少实现过程中出现的混乱和返工。

该角色的核心职责与目标

我认为开发负责人的核心职责主要包括以下几个方面。首先，是参与技术实现方案的落地，协助团队把需求拆解成具体模块、接口和开发任务。其次，是关注代码规范和开发流程，保证团队成员在实现过程中遵循统一的代码风格、目录结构和协作方式。再次，是在开发阶段承担核心功能模块的实现和协调工作，及时发现实现中的问题，并推动团队解决。除了写代码本身，开发负责人还需要兼顾代码质量、可维护性以及团队协作效率。这个角色的核心目标，是保证项目能够稳定、高效地进入实现阶段，并让核心功能真正落地。对我来说，开发负责人最重要的不是单纯完成自己的代码任务，而是帮助团队建立清晰的开发节奏，把需求和设计转化为高质量、可运行、可迭代的系统。

测试负责人（负责测试策略、测试用例设计、质量保障）

刘映池

对所承担角色的理解

我认为测试负责人是帮助项目稳健构筑、模块持续集成的重要推手。这个角色的作用不只是为完成代码的正确性做检验，更需要从代码质量、代码效率、代码能否胜任最终实际需求等多角度评估并给出对代码的修改意见，从而保证最终项目能够符合预期。

该角色的核心职责与目标

在一个复杂的工程项目中，项目各部分功能密不可分。我认为项目工程中测试负责人的核心职责是首先将这些功能尽可能解耦为多个板块，并且为这些不同的板块分别编写单元测试通过单元测试后，再进一步在更高的层次进行整体上的集成测试和系统测试确保各部分的功能能够顺利协作，最后再通过验收测试来验证软件是否满足现实需求。通过这样从局部再到整体的全面检测，项目能够以更加稳定的姿态持续优化、不断更进。因此，测试负责人的核心目标就是用较小的成本，将软件发布后出现重大故障的概率降至可接受的范围内，同时通过技术支持与管理策略建立起团队持续预防项目故障的能力，从而确保最终交付物质量、可靠性和安全性。

二 AI 工具链选型

AI 编程助手

选型工具：Github Copilot、通义灵码

选型理由

- 团队熟悉度高：大部分成员已有使用经验，学习成本为零
- 性价比最优：Github Copilot个人价格在同类工具中最低，且提供免费试用，通义灵码免费使用
- 生态完整：GitHub Copilot与GitHub深度集成，代码补全响应速度快（800-1200ms），适合日常高频使用，通义灵码在Vs Code中有插件集成，方便日常编码使用
- IDE兼容性：支持团队使用的VS Code和IntelliJ IDEA

使用场景

- 可用于生成后端RESTful Controller、Service、MyBatis查询语句的模板代码

- 生成简单的前端Vue组件、请求封装、Element Plus
- 辅助生成简单JUnit / Vitest测试用例

AI 对话工具

选型工具：ChatGPT、Deepseek、Gemini

选型理由

- 这些大模型综合能力较强，适合需求分析、代码解释、Bug排查、SQL生成等多方面任务，能够比较好地帮助我们解决项目中出现的大多数问题
- ChatGPT和Gemini 支持长上下文（如分析整个README文件或多个代码文件），Deepseek对中文理解和生成更自然，适合写用户手册

使用场景

- 需求分析与Web应用核心功能拆解
- 辅助生成API文档/README文件
- 解释报错信息，帮助我们解决一些配置问题
- 更多用户故事与测试用例生成

AI 代码审查工具

选型工具：CodeRabbit、cubic

选型理由

- CodeRabbit适配 Spring Boot 技术栈：支持 Java 静态分析和 AST 解析，能有效识别空指针、事务边界、并发等常见后端问题
- CodeRabbit提供多平台支持，集成 GitHub、GitLab、Azure DevOps，与团队版本控制平台兼容
- cubic 在业务逻辑验证和架构一致性检查方面表现更优，但价格相同且无公开仓库免费选项。若后续发现CodeRabbit 无法满足业务验证需求，可考虑暂时性切换。

使用场景

- 检查前端后端代码中存在的逻辑漏洞
- 检查后端操作数据库的函数是否原子化每次操作
- 对AI辅助生成的代码进行初步审查和筛选

AI 测试工具

选型工具：Qodo（单元测试）+ Keploy（API/集成测试）

选型理由

- Qodo通过IDE 集成支持，可以直接在 IntelliJ IDEA 中生成 JUnit 测试，与后端开发流程无缝衔接；同时Qodo具有上下文感知功能，可以主动基于代码上下文分析生成有意义的边界条件和异常场景测试；并且团队成员可各自使用免费账户，无额外成本
- Keploy完全开源免费，它通过捕获实际 API 流量生成测试用例，适合 CampusHub 的 REST API 测试场景，并且Keploy自动生成测试功能与现有流程契合：可集成到 CI/CD 流水线中自动生成回归测试

使用场景

- 根据代码分板块为项目生成比较基本的单元测试
- 集成到流水线中进行更加随机的测试
- 分析优化已存在的测试代码，找到边界情况进行测试